

Product Requirements Document — NativeID ROS

Product: NativeID ROS (Restaurant Operating System) **Internal codename:** Haven 11 / 702 ROS
Document version: 1.12 **Status:** Living document — describes the current prototype and the target product **Last updated:** 2026-05-23 **Owner:** Product **Source of truth:** `haven11-nextjs/` (Next.js application)

1. Document control

Field	Value
Audience	Product, Engineering, Design, QA, founding operators
Related artefacts	<code>CLAUDE.md</code> (engineering guide), application source under <code>haven11-nextjs/</code>
Review cadence	Update on every module-level change

1.1 Revision history

Version	Date	Summary
1.0	2026-05-22	Initial PRD, reverse-engineered from the working prototype.
1.1	2026-05-22	Implemented the 2026-05-22 stakeholder review: module renames (Front of House , Front of House on Shift , Procurement), order On hold / Closed statuses, the payment-completes-order rule, shift history & configurable shift periods, and a Juice Bar inventory section. All changes are live in the application. Detail in §20.
1.3	2026-05-22	Multi-location inventory (branch Main Store + Kitchen / Bar / Juice Bar sub-stores, one shared catalogue), internal stock requests, Strong Room issue receipts, purchase-order management approval, and a 7th Accountant role approving petty cash. Detail in §21.
1.4	2026-05-23	Staff lifecycle — onboard / offboard (status retained, never deleted), weekly scheduling (Mon–Sun shift assignment), and a new HR Dashboard with KPIs, today's roster, and the week grid. Detail in §22.
1.5	2026-05-23	UX pass — new HR Officer role (own login), file upload for compliance documents (guarantor / contract / ID card), redesigned schedule editor (chip picker + quick-fill presets, à la Deputy / When I Work), and a lower-density list/dashboard layout for /staff and /hr. Detail in §23.
1.6	2026-05-23	Kitchen prep workflow — dual-unit tracking on inventory items (e.g. yam in kg + pcs), photo capture for waste records (peels, breakages), "Prep trim / peel" waste reason, and a Record waste button on Kitchen home. Detail in §24.

Version	Date	Summary
1.7	2026-05-23	UI-level no-duplicates safeguard on the inventory catalogue: the New product dialog is disabled on Kitchen / Bar / Juice Bar tabs (sub-stores are stocked from the Main Store, never define new SKUs) and blocks duplicate names case-insensitively across the catalogue, pointing the user to the existing record. The dialog also exposes the optional alt unit at creation time. Detail in §24.1.
1.8	2026-05-23	No cross-branch requesting — the PO "Deliver to" dropdown is replaced with a read-only label showing the current branch. A PO always delivers to the branch the user is viewing; to order for another branch they switch branches first. Combined with transfers (already pinned to current branch) and internal stock requests (already branch-scoped), no path remains for one branch to request products into another. See FR-PO-9.
1.9	2026-05-23	Categories + bulk-request picker. New <code>category</code> field on every product (14 values from Protein to Cleaning), filterable in the inventory list, shown as a clickable badge per row, and matched by the free-text search. The Internal Stock Request modal is rewritten as a multi-select checklist — search box, category chips, "tick all visible" bulk action, per-row checkbox + qty — so a 50-item request is one screen of work instead of fifty "+ Add row" clicks. See FR-INV-12 / FR-INV-13. Schema bump (v17).
1.10	2026-05-23	Categories are now manageable at runtime. Promoted from a hardcoded <code>const</code> to <code>state.inventoryCategories</code> with <code>addCategory</code> / <code>renameCategory</code> / <code>removeCategory</code> actions (rename cascades to every product; delete blocked while in use). A focused Manage categories modal is reached from two unobtrusive entry points — the inventory Filter popover footer and a <code>Manage...</code> link beside the Category dropdown in the New product dialog — both owner/manager-gated. No new nav item. See FR-INV-14. Schema bump (v18).
1.11	2026-05-23	Transfer actions are now context-aware. Approve / Reject / Issue only render when the viewer is currently in the source branch (the Strong Room for branch requests); the destination side sees an italic status hint instead of action buttons. This prevents a branch from rejecting (or approving, or issuing) its own request. Receive remains the destination-side action; Receipt is visible to both. See FR-TRF-6.
1.12	2026-05-23	Strong Room transfer-request modal upgraded to the multi-select picker — same UX as the internal stock-request picker (search, category chips, tick-and-qty, grouped list, " <i>Tick all visible</i> ", live ready/total counts). Each row shows live Strong Room availability so the requester knows what's actually in stock. Same seamless flow for a 50-item request as for an internal sub-store request. See FR-TRF-2.

Note on accuracy. This PRD is reverse-engineered from the working prototype. Where the prototype simulates a behaviour that production must implement properly (e.g. authentication, persistence), the requirement is written for the *target product* and the prototype's shortcut is called out in §15 (Current state) and §16 (Path to production).

2. Executive summary

NativeID ROS is a **multi-branch Restaurant Operating System** built for a Nigerian hospitality group operating several outlets in Lagos plus a central warehouse. It unifies — in one role-aware application

— front-of-house order taking, kitchen and bar order routing, inventory and recipe costing, inter-branch stock transfers, procurement, petty cash, HR and payroll, delivery dispatch, customer relationship management, events/banqueting, analytics, and an immutable audit trail.

The product is designed around a **hub-and-spoke operating model**: a central **Strong Room** holds bulk stock and runs procurement; **branches** (Lekki, Ikoyi, Agungi) sell to customers and pull stock from the hub via tracked transfers. Every role sees a tailored workspace, every sensitive action is logged, and money/stock leakage is surfaced continuously.

The current build is a **high-fidelity functional prototype**: all behaviour is real (orders deduct stock, shifts reconcile, payroll computes), but data lives in the browser (React state + `localStorage`) rather than a backend. The prototype is feature-complete enough to serve as the functional specification for a production build.

3. Product vision & goals

3.1 Vision

Give an owner-operator of a growing restaurant group a **single, trustworthy system of record** that makes every branch run the same way, makes leakage visible the moment it happens, and lets the owner manage by exception from anywhere.

3.2 Goals

#	Goal	How the product serves it
G1	Eliminate revenue and stock leakage	Recipe-linked stock depletion, blind receiving, over-pour detection, shift reconciliation, waste logging, variance reporting
G2	Standardise multi-branch operations	One app, identical workflows per role, central Strong Room governing supply
G3	Enforce accountability	PIN-gated overrides, manager approvals, immutable audit trail of every sensitive action
G4	Manage by exception	Role dashboards that surface only what needs a decision; live alerts; "needs your decision" queues
G5	Run the full back office	Procurement, petty cash, HR, payroll, fleet — not just front-of-house POS
G6	Make data actionable	Menu engineering, channel mix, peak-hour staffing, leakage %, branch comparison, morning brief

3.3 Non-goals (current release)

- Not a public-facing ordering site or customer mobile app.
- Not an accounting/GL system (it produces inputs for accounting, not statutory books).
- Not a reservations/table-booking product for the public.
- No hardware integration (printers, cash drawers, card terminals, scales) in the prototype.

4. Background & problem statement

Independent and small-group restaurants in the Nigerian market typically run on a patchwork of a basic POS, spreadsheets, WhatsApp, and paper waybills. The consequences:

- **Stock leakage is invisible.** Bar over-pouring, kitchen waste, and theft are not measured until month-end stock-takes, if ever.
- **Branches drift.** Each outlet improvises its own process; the owner cannot compare like-for-like.
- **Cash is unaccountable.** Petty cash and till floats are reconciled informally; shortages are absorbed silently.
- **Procurement is opaque.** Supplier price creep, partial deliveries, and unpaid invoices are tracked in someone's head.
- **No audit trail.** Voids, discounts, and overrides leave no record, so abuse is undetectable.
- **The owner is a bottleneck.** Approvals happen by phone call; the owner has no remote, real-time view.

NativeID ROS addresses these by making the **operating process itself the software** — every transfer, count, void, expense, and payroll run is a structured, logged event scoped to a branch and an actor.

5. Users, roles & personas

The system defines **six staff roles**. Each role has a default landing route, a filtered navigation set, and a branch assignment. The **Owner** is the only role that can switch between branches; every other role is pinned to its home branch.

5.1 Role catalogue

Role	Persona	Home branch	Default route	Primary jobs
owner	Seun O. — Managing Director	All (roaming)	/ Overview	Group oversight, cross-branch comparison, high-value approvals, menu & events strategy
manager	Tunde A. — Branch Manager	Lekki	/manager-dashboard	Run a branch, approve transfers/expenses, manage staff & payroll, monitor leakage
cashier	Ada O. — Cashier	Lekki	/cashier-home	Take orders, run a till shift, handle deliveries & petty cash
kitchen	Amara K. — Head Chef	Lekki	/kitchen-home	Work the kitchen ticket queue
bartender	Chukwu B. — Bartender	Lekki	/bar-home	Work the bar ticket queue, run a bar shift, log waste

Role	Persona	Home branch	Default route	Primary jobs
storekeeper	Eze M. — Storekeeper	Strong Room	/store-home	Manage central inventory, issue transfers, receive purchase orders
accountant	Bola F. — Accountant	All (roaming)	/expenses	Approve petty cash, monitor expense leakage and accounts payable
hr	Funke I. — HR Officer	All (roaming)	/hr	Run HR — onboarding, offboarding, scheduling, compliance documents, payroll oversight

5.2 Persona detail

The Owner (Seun) — wants to know, in 30 seconds, how every branch did today and what needs a decision. Spends time in Overview, Analytics, Audit Trail, Alerts. Approves discounts/expenses above branch limits. Roams branches via the branch switcher.

The Branch Manager (Tunde) — runs Lekki day-to-day. Lives in the Manager Dashboard "needs your decision" queue: approving stock transfers and petty cash, watching alerts, logging staff incidents, running payroll. Cannot see other branches' data.

The Cashier (Ada) — opens a till shift with a float, takes dine-in/takeout/delivery orders on the POS, charges via cash/card/transfer/split, and closes the shift by counting the drawer. Voids require a manager PIN.

Kitchen & Bar staff (Amara, Chukwu) — work a single screen: a live queue of tickets that they bump through New → Preparing → Ready → cleared. The bartender additionally runs a shift and logs waste.

The Storekeeper (Eze) — sits at the Strong Room. Receives supplier deliveries into central stock, approves and issues branch transfer requests, generates waybills.

6. Scope

6.1 In scope (delivered in the prototype)

Authentication & role routing; six role workspaces; Front of House (order taking) with floor plan; Kitchen/Bar display; Inventory with batches & waste; Stock Transfers (hub-and-spoke); Purchase Orders & Vendors; Menu & Recipes; Expenses & Petty Cash; Staff/HR; Payroll; Dispatch & Fleet; CRM (customers, feedback, complaints); Events/BEO; Analytics; Audit Trail; Alerts; multi-branch model; CSV import/export.

6.2 Out of scope (this release)

Real backend/database; real authentication & secrets management; hardware peripherals; payment-gateway integration; offline mode; native mobile apps; multi-currency; multi-tenant (multiple restaurant groups); statutory accounting exports; public ordering.

7. System context & architecture

7.1 Technology stack

Layer	Choice
Framework	Next.js 15.5 (App Router)
UI runtime	React 19
Language	TypeScript 5.8
Styling	Tailwind CSS v4 (configured in <code>app/globals.css</code> , <code>oklch</code> token palette)
Component kit	shadcn/ui (new-york style), Radix primitives, lucide-react icons
Charts	Recharts (available); most charts hand-rolled with divs
Toasts	Sonner
Font	Outfit (Google Fonts)
Hosting	Vercel (framework preset must be Next.js ; see <code>vercel.json</code>)

7.2 Application architecture

- `AuthProvider` (`lib/auth.tsx`) — client-side session. Holds the current `StaffUser` , exposes `login / logout` . Session persisted in `localStorage` key `nativeid_user` .
- `StoreProvider` (`lib/store.tsx`) — the entire domain model and business logic in one React context: ~30 entity collections plus ~60 mutation functions. State persisted in `localStorage` key `haven11_store_v11` , seeded with realistic demo data on first load.
- `AuthGuard` — wraps the app; redirects unauthenticated users to `/login` , shows a splash while resolving.
- `AppShell` — every page renders inside it: role-filtered sidebar, sticky header, branch switcher, notification bell. Also exports the `PageSection` (card) and `Stat` (KPI tile) primitives.
- **Pages** — one route per module under `app/` . Each page reads from `useStore()` / `useAuth()` and calls store mutations.

7.3 The data seam

All data and logic currently sit in `lib/store.tsx` . **The production migration path** is to replace the in-context state and mutation bodies with API calls to a real backend, keeping the `useStore()` hook surface (the ~60 named functions) stable so pages do not change. This is the single most important architectural seam.

8. Information architecture & navigation

8.1 Route → role access matrix

Navigation is filtered per role by `roleNav` in `AppShell.tsx`. ✓ = module appears in that role's sidebar.

Route	Module	owner	manager	cashier	kitchen	bartender	storekeeper
/	Operations Overview	✓					
/manager-dashboard	Branch Dashboard		✓				
/cashier-home	Cashier — My Shift			✓			
/kitchen-home	Kitchen Display				✓		
/bar-home	Bar Queue					✓	
/store-home	Store Home						✓
/pos	Front of House	✓	✓	✓		✓	
/kitchen-bar	Kitchen & Bar Display	✓	✓				
/inventory	Inventory	✓	✓				✓
/transfers	Stock Transfers	✓	✓				✓
/purchase-orders	Procurement	✓	✓				✓
/vendors	Vendors	✓	✓				✓
/menu	Menu & Recipes	✓					
/dispatch	Dispatch & Fleet	✓	✓	✓			
/cashier	Front of House on Shift	✓	✓				
/expenses	Expenses & Petty Cash	✓	✓	✓			✓
/staff	Staff (HR)	✓	✓				
/hr	HR Dashboard	✓	✓				
/payroll	Payroll	✓	✓				
/events	Events / BEO	✓					
/customers	Customers (CRM)	✓	✓				
/reports	Analytics	✓	✓				

Route	Module	owner	manager	cashier	kitchen	bartender	storekeeper
/audit	Audit Trail	✓	✓				
/alerts	Alerts & Security	✓					

The **Accountant** (added in v1.3) is a roaming finance role — it sees **Expenses, Procurement, Vendors** and the **Audit Trail**. The **HR Officer** (added in v1.5) is a roaming HR role — sees **HR Dashboard, Staff, Payroll** and the **Audit Trail**.

Known gap (security): AuthGuard only enforces *authenticated vs not*. It does **not** hard-gate routes by role — the matrix above is enforced only by hiding sidebar links. A signed-in user who types another route directly is not blocked. Production must enforce this matrix server-side. See FR-AUTH-5 and §17.

8.2 Global shell elements

- **Sidebar** — branded header, role-filtered nav, current-user card, sign out. Collapses to a drawer on mobile.
- **Header** — page title/subtitle, **branch switcher**, **notification bell**.
- **Branch switcher** — owner sees a dropdown of all branches; all other roles see a static chip of their home branch.
- **Notification bell** — live count + dropdown derived from the current branch's state: over-pour incidents, expiring batches (≤ 14 days), and low/out-of-stock items. Items deep-link to `/alerts` or `/inventory`.

9. Domain model

The store defines the following core entities (see `lib/store.tsx` for full field lists).

Entity	Purpose / key fields
Branch	<code>id</code> , <code>name</code> , <code>kind</code> (<code>hub</code> <code>branch</code>). Four seeded: Strong Room (hub), Lekki, Ikoyi, Agungi.
InventoryItem	Stock row keyed by branch + location + SKU: <code>sku</code> , <code>branch</code> , <code>location</code> (<code>store</code> / <code>kitchen</code> / <code>bar</code> / <code>juice-bar</code>), <code>onHand</code> , <code>reorder</code> (par level), <code>unit</code> , <code>cost</code> , <code>line</code> . Optional <code>altUnit</code> + <code>altOnHand</code> capture a second measurement (e.g. 30 kg yam recorded alongside 10 pcs). One shared product catalogue — no duplicate SKUs across locations.
MenuItem	<code>name</code> , <code>category</code> , <code>price</code> , <code>emoji</code> , <code>status</code> (Available/Sold out/Coming soon), <code>recipe</code> (SKU + qty lines).
Order	Sale: <code>branch</code> , <code>table</code> , <code>channel</code> , <code>customer</code> , <code>lines</code> , <code>subtotal</code> , <code>discount</code> , <code>vat</code> (7.5%), <code>total</code> , <code>payments</code> , <code>splitWays</code> , <code>staffName</code> , <code>shiftId</code> , <code>status</code> (On hold / Closed), <code>voided</code> . An order is <code>Closed</code> only once payment is received.

Entity	Purpose / key fields
Shift	Till/bar session: <code>staffId</code> , <code>role</code> , <code>branch</code> , <code>openingFloat</code> , <code>countedCash</code> , <code>openedAt</code> / <code>closedAt</code> , <code>status</code> .
Ticket	Kitchen/Bar work item: <code>station</code> , <code>label</code> , <code>items</code> , <code>status</code> (New/Preparing/Ready).
TableRec	Floor plan unit: <code>label</code> , <code>zone</code> , <code>seats</code> , <code>status</code> , <code>guests</code> , <code>orderTotal</code> .
Transfer	Inter-branch waybill: <code>fromBranch</code> (always hub), <code>toBranch</code> , <code>lines</code> (requested/issued/received qty), <code>status</code> , <code>valueAtCost</code> . An issued transfer carries a printable receipt.
StockRequest	Internal request to move stock from a branch Main Store into a Kitchen / Bar / Juice Bar sub-store: <code>toLocation</code> , <code>lines</code> , <code>status</code> (Requested/Issued).
Vendor	Supplier: <code>name</code> , <code>contact</code> , <code>terms</code> (COD/Net 15/Net 30), <code>tin</code> , <code>category</code> .
PurchaseOrder	Procurement: <code>vendorId</code> , <code>branch</code> (destination), <code>lines</code> , <code>status</code> (Pending Approval → Ordered → Partially/Received, or Rejected), <code>expectedDate</code> , <code>total</code> , <code>paid</code> .
Batch	Perishable lot with <code>expiry</code> date, for FIFO/expiry tracking.
PriceChange	Logged supplier cost variance (old → new).
WasteEntry / StockCount	Inventory loss & variance records (count carries an <code>overPour</code> flag). <code>WasteEntry</code> may include an attached photo (filename + data URL for small files) captured on the prep floor.
ExpenseRequest / Wallet	Petty cash requisition lifecycle + per-branch imprest wallet.
Employee / Attendance / Disciplinary	HR records, clock-ins with lateness, incidents & commendations. <code>Employee.status</code> is Active / Suspended / Offboarded (offboarding never deletes); <code>Employee.weeklySchedule</code> (Mon–Sun) drives the roster. <code>Employee.compliance</code> holds three uploaded <code>ComplianceDoc</code> s — guarantor form, signed contract, ID card — each with filename, size, upload date and a downloadable data URL for small files.
PayrollRun / Payslip	Monthly payroll with statutory deductions.
Rider / DeliveryJob	Fleet members and delivery jobs (COD tracking).
Customer / Feedback / ComplaintTicket	CRM golden record (keyed by phone), ratings, complaints.
RestaurantEvent	Banquet/event booking with cost lines, deposit, P&L.
AuditEntry	Immutable who/what/when/where/amount record; <code>category</code> , <code>severity</code> .

9.1 Reference data & constants

- **Branches:** Strong Room (`strong-room` , hub), Lekki, Ikoyi, Agungi.

- **Inventory locations:** each branch holds stock in a **Main Store** plus **Kitchen, Bar** and **Juice Bar** sub-stores; the Strong Room has a single store. `line` (Kitchen/Bar/Lounge/Juice Bar) categorises a product; `location` is where the stock physically sits.
 - **VAT:** 7.5% on (subtotal – discount).
 - **Payroll statutory rates:** PAYE 8% of gross; Pension 8% of base; NHF 2.5% of base; lateness ₦500 per late mark (≥15 min).
 - **Currency:** Nigerian Naira (₦). All amounts integer Naira.
-

10. Cross-cutting requirements

These behaviours apply across all modules.

10.1 Multi-branch operating model

- **FR-BR-1** Every stock row, order, shift, transfer, expense, employee, and audit entry is scoped to a `branch`.
- **FR-BR-2** The **Owner** may switch the operating branch via the header switcher; all data re-scopes immediately.
- **FR-BR-3** All non-owner roles are **pinned** to their home branch; `AppShell` forces `currentBranch` back to the user's branch on every render.
- **FR-BR-4** The **Strong Room** is a hub: it holds bulk stock and is the sole `fromBranch` of every transfer. Branches receive stock only via transfers from the hub or via direct PO receipt.

10.2 Role-based access control (RBAC)

- **FR-RBAC-1** Sidebar navigation is filtered to the role's `roleNav` set.
- **FR-RBAC-2** Owner sees group-wide data on shared modules (Audit, Reports, Cashier); manager and below see only their branch.
- **FR-RBAC-3** (*Production*) Route access must be enforced server-side per the §8.1 matrix, not only by hiding links.

10.3 Immutable audit trail

- **FR-AUD-1** The store automatically writes an `AuditEntry` for every sensitive action: order void, discount applied, shift close, stock variance/over-pour, waste, transfer approve/reject/issue/receive, goods received, cost-price change, expense approve/reject/disburse/reconcile, wallet top-up, disciplinary, payroll run.
- **FR-AUD-2** Each entry records `actor`, `branch`, `category`, `action`, human-readable `detail`, optional `ref` and `amount`, and `severity` (info/warning).
- **FR-AUD-3** Entries cannot be edited or deleted from the UI.

10.4 Approvals & PIN gating

- **FR-APR-1** A `ManagerApprovalModal` validates a 4-digit PIN against the staff roster; an `ownerOnly` flag restricts to the owner.
- **FR-APR-2** POS discounts above ₦500 require a manager/owner PIN.
- **FR-APR-3** Order voids require manager approval and a reason.
- **FR-APR-4** Petty cash approval is tiered by amount (see §11.13).

10.5 Notifications & alerts

- **FR-NOT-1** The header bell shows a live unread count derived from current-branch state (over-pours, expiring batches, low/out stock).
- **FR-NOT-2** Alerts deep-link to the relevant module; "mark all read" clears the count (session-local).

10.6 Data import / export

- **FR-IO-1** Inventory and Menu support **bulk CSV import** (upload or paste) via `ImportModal`, with a downloadable template, tolerant column matching, and a live preview. Existing rows update; new rows insert.
- **FR-IO-2** Inventory, Analytics (menu engineering), Payroll (bank schedule), and Audit Trail support **CSV export** of the current filtered view.

10.7 Persistence

- **FR-PER-1** (*Prototype*) All state persists to `localStorage` and survives refresh; a `resetAll` restores seed data.
- **FR-PER-2** (*Production*) State must persist to a backend database with concurrent multi-device access.

10.8 Design system

- **FR-DS-1** All pages use `AppShell` + `PageSection` + `Stat` for layout consistency.
 - **FR-DS-2** Status colours are semantic and consistent: green = OK/positive, amber `text-warning` = caution/low, red `destructive` = critical/loss, deep-green `primary` = active/CTA.
 - **FR-DS-3** Fully responsive; sidebar becomes a drawer below `lg`.
-

11. Functional requirements by module

Each module below lists its purpose, primary users, and numbered functional requirements.

11.1 Authentication & login (`/login`)

Users: all. **Purpose:** identify the staff member and route them to their workspace.

- **FR-AUTH-1** The login screen shows the staff roster as selectable avatar cards, each with a role-coloured icon ("Who's working today?").

- **FR-AUTH-2** Selecting a staff member opens a 4-digit PIN pad; the PIN auto-submits on the 4th digit.
- **FR-AUTH-3** A correct PIN starts a session and redirects to the role's `defaultRoute`; a wrong PIN shows a shake animation and error, and clears the entry.
- **FR-AUTH-4** A session persists across refreshes; an already-authenticated user is redirected away from `/login`.
- **FR-AUTH-5** (*Production*) Replace PINs with proper credentials/secrets, hash-verified server-side; enforce route-level RBAC; support session expiry and idle auto-lock (the Alerts page already advertises "idle sessions auto-locked").

11.2 Operations Overview — Owner dashboard (/)

Users: owner. **Purpose:** 30-second read on a branch's live trading.

- **FR-OVW-1** KPI tiles: **Sales today** (¥ + order count), **Covers seated**, **Open tickets** (kitchen+bar), **Stock alerts** (amber when > 0).
- **FR-OVW-2 Revenue by channel** panel — Dine-in / Takeout / Delivery split with amounts, %, and bars.
- **FR-OVW-3 Live alerts** panel — top over-pours and low-stock items; each deep-links to `/alerts`.
- **FR-OVW-4 Recent orders** table — last 6 orders with channel, item count, total, age.
- **FR-OVW-5 Inventory health** — per line (Kitchen/Bar/Lounge) a stacked OK/Low/Out bar with counts.
- **FR-OVW-6** All figures exclude voided orders and re-scope when the owner switches branch.

11.3 Branch Dashboard — Manager (/manager-dashboard)

Users: manager. **Purpose:** run a branch and clear the decision queue.

- **FR-MGR-1** KPI tiles: Sales today, Open tickets, **Pending approvals** (transfers + expenses, amber when > 0), Stock alerts.
- **FR-MGR-2 Needs your decision** — combined queue of pending transfers and pending expenses; each item deep-links to its approval page.
- **FR-MGR-3 Live alerts** panel (over-pours + low stock).
- **FR-MGR-4 Recent orders** list (last 6).
- **FR-MGR-5 Quick links** to POS, Stock Transfers, Inventory, Expenses.

11.4 Role home screens

Lightweight landing pages for operational roles.

- **FR-HOME-1 — Cashier home (/cashier-home)**: shift banner; large "New Order" CTA → `/pos`; shift KPIs (Cash/Card/Transfer totals + order count); floor plan with seat/clear-table actions; recent orders list with **void** (manager-approved).
- **FR-HOME-2 — Kitchen home (/kitchen-home)**: status counts (Waiting/Cooking/Ready); large ticket cards; each card advances New → Preparing → Ready → cleared; tickets ≥ 12 min flagged overdue.

- **FR-HOME-3 — Bar home (/bar-home)**: shift banner; "Record waste" action; status counts (Waiting/Making/Ready); ticket cards with the same advance flow; tickets ≥ 8 min flagged slow.
- **FR-HOME-4 — Store home (/store-home)**: task aggregator — low stock, transfers to issue/receive, POs to receive, batches expiring ≤ 14 days; a count of "tasks that need you today"; CTA to raise a transfer.

11.5 Front of House (/pos)

Users: owner, manager, cashier, bartender. **Purpose:** take orders, run open tabs, and take payment.

Renamed in v1.1. Formerly "Point of Sale (POS)" — renamed to **Front of House (FOH)** because "POS" is widely read as the physical card/payment terminal, which confused users. The order-taking screen is labelled Front of House; the route stays /pos .

- **FR-POS-1 — Service selection:** start a **Dine-in** order by tapping an available table on the zoned floor plan (Indoor/Terrace/Bar), or a **Takeout / Delivery** order via CTA cards. Occupied/reserved tables are not selectable.
- **FR-POS-2 — Seat guests:** seating a table captures guest count (bounded by table capacity) and marks the table occupied.
- **FR-POS-3 — Service forms:** Takeout captures customer name, phone, pickup time (ASAP / 15 min / 30 min / 1 hr); Delivery captures name, phone, address, and a delivery fee (default ₦1,500).
- **FR-POS-4 — Menu & cart:** category-tabbed menu grid (Available items only); tapping an item adds/increments a cart line; per-line quantity $\pm$ and delete; sticky cart sidebar shows order label and context.
- **FR-POS-5 — Modifiers:** each line can take modifiers (Extra cheese +~~₦~~300, Extra sauce +~~₦~~200, Extra protein +~~₦~~800, Extra spicy, No onions, Well done) and a free-text instruction; both print on the kitchen/bar ticket.
- **FR-POS-6 — Discounts:** apply a percentage or fixed-amount discount with a reason; discounts over ₦500 require a manager/owner PIN; the discount is written to the audit trail.
- **FR-POS-7 — Totals:** cart shows Subtotal, Discount, VAT 7.5%, Total.
- **FR-POS-8 — Payment:** split the bill 1–8 ways; take one or more payments across **Cash / Card / Transfer**; quick-fill "remaining" and "one share"; show paid / still-due / change.
- **FR-POS-9 — Cash on delivery:** delivery orders may complete as COD (rider collects total + fee).
- **FR-POS-10 — Completing a sale (recordSale)** atomically: creates the `Order` ; deducts each recipe ingredient from **this branch's** stock; raises Kitchen and/or Bar tickets (drinks/cocktails route to Bar); for delivery, raises a `DeliveryJob` ; for takeout/delivery, upserts the customer golden record by phone; links the order to the cashier's open shift; logs a discount audit entry if applicable.
- **FR-POS-11 — Confirmation:** a done screen shows order number, payment breakdown, change, and customer details; "New order" resets.
- **FR-POS-12 — Open a tab:** an **Open tab** action places the order as **On hold** — items fire to the kitchen/bar and stock is deducted immediately, with no payment taken. The order joins the **"Open tabs"** list on the Front of House start screen (seated and delivery alike).

- **FR-POS-13 — Amend an open tab:** tapping an open tab resumes it — items already sent to the kitchen are shown locked; the cashier adds more items and either **sends the new items to the kitchen** (the tab stays open) or proceeds to close it. Newly-added items fire fresh kitchen/bar tickets and deduct their stock; the tab total recomputes.
- **FR-POS-14 — Close & pay:** closing a tab takes payment (cash or POS/card terminal); the order becomes **Closed**, a dine-in table is freed, and only then does it count as a sale. On-hold tabs are excluded from revenue on dashboards and analytics.

11.6 Kitchen & Bar Display (/kitchen-bar , plus role homes)

Users: owner, manager (combined view); kitchen/bartender (single-station homes).

- **FR-KDS-1** Tickets are generated by Front of House and routed by station — drinks & cocktails to **Bar**, everything else to **Kitchen**.
- **FR-KDS-2** Each ticket shows order ref, table/channel label, item lines with modifiers/notes, status, and age.
- **FR-KDS-3** Staff advance a ticket New → Preparing → Ready; "serve & clear" removes a Ready ticket; "Mark ready" jumps straight to Ready.
- **FR-KDS-4** Ticket age is colour-flagged (kitchen overdue ≥12 min; bar slow ≥8 min).
- **FR-KDS-5** Voiding the source order removes its tickets.

11.7 Front of House on Shift (/cashier) + shift lifecycle

Users: owner, manager (ledger view); cashier/waiter/bartender (own shift via `ShiftBanner`).

Renamed in v1.1. Formerly "Cashier & Shifts" — renamed to **Front of House on Shift** because the module serves **waiters as well as cashiers**: anyone who works an opening balance and handles cash.

- **FR-SHF-1** A cashier/bartender opens a shift with an **opening cash float** (default ₦50,000).
- **FR-SHF-2** Orders rung during a shift link to it; the shift banner shows elapsed time, order count, sales, and waste.
- **FR-SHF-3** Closing a shift captures **counted cash**; the system computes variance = counted – (float + sales) and writes a "Shift closed" audit entry (warning if short).
- **FR-SHF-4** A **bartender** closing a shift additionally performs a **bar stock count** per Bar SKU; negative variances are flagged as **over-pours** attributed to the bartender.
- **FR-SHF-5** The `/cashier` page is the **live reconciliation ledger**: per shift — staff member, date, opened time, shift period, counted cash, variance (colour-coded), status; KPIs for cash in tills, reconciled variance, open shifts. Branch-scoped (owner sees all).
- **FR-SHF-6 — Shift history:** the ledger shows each shift's **date** and offers a **date drop-down** to review past days, not just today.
- **FR-SHF-7 — Configurable shift periods:** on clock-in, staff pick a **shift period** — **Full day** (single-shift branch) or **1st / 2nd / 3rd shift** (multi-shift branch); the period shows on the ledger.

11.8 Inventory (/inventory)

Users: owner, manager, storekeeper. **Purpose:** branch stock control.

- **FR-INV-1** KPI tiles: Total SKUs, Low stock (click to filter), Out of stock (click to filter), Stock value (Σ onHand $\times$ cost).
- **FR-INV-2** Searchable, filterable (by line, by status) inventory table; status derived as OK / Low ($\leq$ reorder) / Out (≤ 0).
- **FR-INV-3 New SKU** — add an item with auto-generated SKU (line-prefixed), unit, on-hand, reorder level, cost.
- **FR-INV-4 Adjust** — per item, either **Receive stock** (add quantity) or **Stock count** (set actual; logs a `StockCount` with variance and cost; Bar negative variance = over-pour, attributed to the bartender on shift if any).
- **FR-INV-5 Waste** — log spoilage/loss via `WasteModal` (item, quantity, reason); deducts stock, records cost, attributes to staff/shift.
- **FR-INV-6 Activity log** (toggle) — recent stock-count variances and waste entries.
- **FR-INV-7 Batch & expiry register** — perishable batches sorted soonest-expiry-first, colour-coded (≤ 7 days red, ≤ 14 amber); supports FIFO discipline.
- **FR-INV-8 Import** (bulk CSV upsert) and **Export** (filtered CSV) of inventory.
- **FR-INV-9** All inventory data is scoped to the current branch; Front-of-House sales deduct stock here in real time.
- **FR-INV-10 — Section separation:** inventory carries a **section** — **Kitchen, Bar, Juice Bar**, or Lounge — so each section is filtered, audited, and managed independently, reflecting that sections deplete stock at different rates.
- **FR-INV-11 — Stock locations:** each branch keeps stock in a **Main Store** plus **Kitchen, Bar** and **Juice Bar** sub-stores. The Inventory page has a location tab per store; counts, waste, receiving and adjustments act on the selected location. A product is defined once (one shared catalogue) — there are no duplicate goods across the Main Store and the sub-stores.
- **FR-INV-12 — Internal stock requests:** the Kitchen, Bar and Juice Bar **request stock from the branch Main Store**; the request is **issued** from the Main Store, moving stock into the requesting sub-store. Front-of-House sales then deduct each recipe ingredient from the sub-store matching its line. The request modal is a **multi-select picker** — the operator sees the Main Store as a single scrollable list grouped by category, with a search box, category chips, a "tick all visible" bulk action, and a per-row checkbox + qty input. Built for 50+-item requests in one screen, not row-by-row.
- **FR-INV-13 — Categories:** every product carries a **category** (Protein, Produce, Grains, Spices, Oils & Fats, Dairy, Beer, Spirits, Wine, Mixers, Hot Drinks, Soft Drinks, Cleaning, Other). The Inventory list shows the category as a clickable badge in the Item cell (click to filter), exposes Category as a chip filter alongside Line and Status, and the free-text search matches name, SKU **and** category — so typing "spice" surfaces all spices, "protein" surfaces all proteins. The stock-request picker groups by the same category vocabulary. CSV import/export carries a `Category` column (an unknown value coerces to "Other" rather than crashing the import).
- **FR-INV-14 — Category management (owner/manager):** the category vocabulary lives in `state.inventoryCategories`, not as a hardcoded constant, and is curated through a focused **Manage categories** modal — add, rename, delete in one list with per-category product counts.

Rename cascades to every stock row using that category. **Delete is blocked** while a category is in use (counter > 0); the operator must reassign products to another category first. The modal is reached from **two unobtrusive entry points** — a `Manage categories` → link in the inventory Filter popover footer, and a `Manage...` link beside the Category dropdown in the **New product** dialog (the Linear/Notion pattern: no new nav item, manage from where you'd create). Cashier, kitchen, bartender, storekeeper, accountant and HR see the categories but cannot edit them. Seeded with the 14 defaults from FR-INV-13.

11.9 Stock Transfers (`/transfers`)

Users: owner, manager, storekeeper. **Purpose:** govern hub→branch stock movement.

- **FR-TRF-1** KPI tiles: Awaiting approval, Approved, In transit, Disputed.
- **FR-TRF-2 Request** — a branch raises a transfer request from the Strong Room with a reason (Low stock / Event prep / Spoilage replacement / Emergency) and line items. The request modal is a **multi-select checklist** matching the internal stock-request picker (FR-INV-12): a single scrollable list of every item in the Strong Room's Main Store grouped by category with sticky headers, a search box (matches name / SKU / category), a category-chip filter, **"Tick all visible"** bulk action, and per-row checkbox + qty input. Each row shows live Strong Room availability so the requester knows what's actually in stock. Footer shows *"X/Y ready · Z total units"* live as items are ticked. Built for 50-item requests in one screen, not row-by-row. (Disabled while the operating context is the hub.)
- **FR-TRF-3** Lifecycle: **Requested** → **Approved/Rejected** → **Issued** → **Received** (or **Disputed**).
- **FR-TRF-4 Issue** — the Strong Room confirms dispatched quantities (default = requested), generates a waybill, deducts hub stock, and bills the destination branch at cost (`valueAtCost`).
- **FR-TRF-5 Receive** — the destination counts what physically arrived; any shortfall vs issued quantity flags the transfer **Disputed** (a goods-in-transit loss escalated for investigation); received stock is added to the destination branch.
- **FR-TRF-6 — Actions are context-aware (source vs destination):** approve / reject / issue are **source-side actions** — they only render when the viewer is currently in the source branch (the Strong Room for branch requests). A branch viewing its own request sees an italic status hint instead of action buttons (*"Awaiting Strong Room approval"* → *"Approved · Strong Room preparing"* → *"In transit · awaiting receipt"*). Receive is the **destination-side action** and the source instead sees *"In transit · awaiting receipt"* while waiting. The Receipt waybill view is available to both sides on Issued / Received / Disputed. This prevents a branch from rejecting (or approving, or issuing) its own request.
- **FR-TRF-6** Every state change writes an audit entry. Non-owners see only transfers involving their branch.
- **FR-TRF-7 — Issue receipt:** the Strong Room sells stock to branches — an issued transfer produces a printable **receipt** (waybill ref, destination, line items with unit cost and totals, grand total at cost).

11.10 Procurement (`/purchase-orders`)

Users: owner, manager, storekeeper (finance sections owner/manager only).

Renamed in v1.1. The module is labelled **Procurement** (formerly "Purchase Orders" / "process order") — clearer for staff whose task is to *order inventory* for the kitchen and bar. A Purchase

Order is the document Procurement produces.

- **FR-PO-1** KPI tiles: Open POs, Awaiting delivery; finance-only: Unpaid invoices, Accounts payable total.
- **FR-PO-2 Create PO** — select a vendor, expected date, and line items (SKU, quantity, unit cost auto-prefilled from catalogue); delivered to the Strong Room.
- **FR-PO-3 Receive goods** — count received quantities with optional **blind receiving** (hides ordered quantities to force honest counts); capture per-line actual unit cost and optional expiry date.
- **FR-PO-4** On receipt: stock is added to the hub; a cost variance vs catalogue creates a **PriceChange** and rolls the cost forward (audit-logged); an expiry date creates a **Batch**; status becomes Partially Received or Received.
- **FR-PO-5 Mark paid** — record invoice payment; status badge updates.
- **FR-PO-6 Accounts payable — aging** — outstanding POs by age (red after 30 days).
- **FR-PO-7 Supplier price changes** — log of received-cost variances with % change.
- **FR-PO-8 — Management approval:** a purchase order is created as **Pending Approval** and must be **approved by management** (owner/manager) before it becomes **Ordered** (sent to the vendor); it can also be **Rejected**. Goods can only be received once approved.
- **FR-PO-9 — Destination is the current branch (no cross-branch requesting):** a PO always delivers to the branch the user is currently viewing — the **Deliver to** field is a read-only label, not a dropdown. To raise a PO for another branch, the user switches branches via the header BranchSwitcher (owners, managers, accountants and HR can roam; branch-scoped roles only ever see their own branch). Combined with **FR-TR-2** (which already pins `toBranch = currentBranch` on new transfers), this means **branches cannot request products to be delivered to other branches** through any path in the app.

11.11 Vendors (/vendors)

Users: owner, manager, storekeeper.

- **FR-VND-1** KPI tiles: Vendors, Outstanding payables, Open POs.
- **FR-VND-2** Vendor database table: name, category, payment terms, contact, amount owed.
- **FR-VND-3 New vendor** — name, contact, phone, email, TIN, terms (COD/Net 7/15/30), category.
- **FR-VND-4 Vendor detail** — contact card, lifetime spend, outstanding balance, and full purchase-order history.

11.12 Menu & Recipes (/menu)

Users: owner.

- **FR-MNU-1** Category-filtered grid of menu items; each card shows price, recipe cost, **profit margin** % (red < 15%), ingredient count, and status.
- **FR-MNU-2 New menu item** — name, category, price, emoji; immediately orderable in the POS.
- **FR-MNU-3 Edit item** — name, price, availability, and a **recipe builder** linking inventory SKUs + quantities; a live summary shows recipe cost vs price vs margin.

- **FR-MNU-4 View recipe** — per-ingredient cost breakdown; recipes are what drive real-time stock depletion on every sale.
- **FR-MNU-5 Bulk import** menu items via CSV (recipes linked afterwards by editing).

11.13 Expenses & Petty Cash (/expenses)

Users: owner, manager, cashier, storekeeper.

- **FR-EXP-1** KPI tiles: Pending approval, Awaiting reconciliation, Spent this period.
- **FR-EXP-2 Branch wallet** — per-branch imprest wallet with balance vs target float; low-balance warning; **request top-up** (reimbursement from Head Office).
- **FR-EXP-3 New request** — category (8 categories, each with a monthly budget), amount, description; the system shows remaining category budget and the approval tier.
- **FR-EXP-4 Accountant approval:** petty-cash requests are approved (or rejected) by the **Accountant** — a dedicated role with its own login and finance workspace. The Owner may also approve.
- **FR-EXP-5** Lifecycle: **Pending** → **Approved/Rejected** → **Disbursed** → **Reconciled**. Disbursement debits the branch wallet.
- **FR-EXP-6 Reconcile** — record actual spend and receipt (or a manager waiver); unspent change is returned to the wallet.
- **FR-EXP-7** A staff member with an open (disbursed, unreconciled) requisition is blocked from raising a new one.
- **FR-EXP-8 Expense leakage** report (manager/owner) — spend vs budget per category, over-budget bars in red.
- **FR-EXP-9** Approve/reject/disburse/reconcile and top-up all write audit entries.

11.14 Staff / HR (/staff)

Users: owner, manager.

- **FR-HR-1** KPI tiles: Active staff, Clock in today, Compliance gaps, Certs expiring (≤ 30 days).
- **FR-HR-2** Employee records table with avatar, role, today's attendance, lateness marks, and compliance/cert flags.
- **FR-HR-3 Onboard staff** — name, role, phone, next of kin, shift start, salary structure (base/transport/housing); created **Active** with compliance docs unchecked and a 1-year food-handler cert.
- **FR-HR-4 Employee file** — contact, compliance documents (guarantor / contract / ID card) with **file upload** (PDF / JPG / PNG, ≤ 5 MB), Replace and Download actions, upload date, filename and size; plus certification status, salary structure, last-5 attendance, disciplinary & commendation history.
- **FR-HR-5 Clock in / out** — clock-in computes lateness vs scheduled start.
- **FR-HR-6 Log incident** — type (Lateness, Customer complaint, Shortage, Misconduct, Damage, Commendation) + description + action; audit-logged.
- **FR-HR-7** A three-strike lateness rule surfaces a "warning due" flag at ≥ 3 late marks.
- **FR-HR-8 — Offboard staff:** an employee can be **Offboarded** with a reason — they no longer count in active staff, the roster, or payroll, but the record (attendance, disciplinary, salary history) is **retained on file, never deleted**. Offboarding is reversible via **Reactivate**.

- **FR-HR-9 — Status filter:** the records table hides offboarded staff by default; a toggle shows them (greyed) for audit and HR review.
- **FR-HR-10 — Weekly schedule:** each employee has a 7-day schedule (Mon–Sun) where each day is assigned a shift period — **Off, Full day, 1st shift, 2nd shift, or 3rd shift**. The editor uses **per-day chip pickers with quick-fill presets** (Mon–Fri 1st shift, Mon–Sat Full day, etc.), colour-coded by shift, and a working-days summary — modeled on Deputy / When I Work. Rolled up into the HR Dashboard roster.

11.15 Payroll (/payroll)

Users: owner, manager.

- **FR-PAY-1** KPI tiles: On payroll, Monthly gross, Last run net, Payroll runs count.
- **FR-PAY-2 Run payroll** for a named period — generates a **Payslip** for every active employee at the branch.
- **FR-PAY-3** Payslip computation: Gross = base + transport + housing; minus PAYE 8% of gross, Pension 8% of base, NHF 2.5% of base; minus lateness deduction ($\text{N}500 \times \text{late marks}$); minus cash-shortage deduction carried from negative shift variances under that employee's name.
- **FR-PAY-4 Payslip view** with full earnings/deductions breakdown and **print**.
- **FR-PAY-5 Export bank schedule** as CSV (employee, role, gross, deductions, net).
- **FR-PAY-6 Run history** of prior payroll runs.

11.16 Dispatch & Fleet (/dispatch)

Users: owner, manager, cashier.

- **FR-DSP-1** KPI tiles: To dispatch, Out for delivery, COD outstanding, Delivered.
- **FR-DSP-2 Active deliveries** board — each delivery job (created from a POS delivery order) shows customer, address, phone, COD/prepaid, fee, age, assigned rider.
- **FR-DSP-3** Delivery lifecycle: **Preparing** → **Ready for pickup** → **Out for delivery** → **Delivered**.
- **FR-DSP-4 Assign rider** — hand a ready job to an internal bike or 3PL partner; COD jobs warn the assigner of the cash to collect.
- **FR-DSP-5 Fleet roster** — riders (internal bike / 3PL) with status, deliveries completed, fee revenue, and net P&L (bikes) or COD held (3PL).
- **FR-DSP-6 Add rider, clock in/out, log fleet expense** (fuel/repair/maintenance — reduces a bike's net P&L), and **settle COD** (a rider cannot clock out while holding unsettled COD cash).

11.17 Customers / CRM (/customers)

Users: owner, manager.

- **FR-CRM-1** KPI tiles: Customers, VIPs, Open complaints, Branch mood (avg feedback rating).
- **FR-CRM-2 Golden-record database** keyed by **phone**; customers are auto-captured from takeout/delivery orders.
- **FR-CRM-3 Customer 360°** — tier control (New/Regular/VIP/Blacklisted), visits, lifetime value, average spend, last seen, favourite item, complaint and feedback history.

- **FR-CRM-4 Log feedback** — food/service/ambience star ratings + comment; a keyword sentiment classifier tags it; a low average (≤ 2) or negative sentiment **auto-raises a complaint ticket**.
- **FR-CRM-5 Complaint tickets** — Open → In progress → Resolved; tickets open > 24h flagged as escalated.
- **FR-CRM-6 Marketing** — win-back list (not seen 30+ days) and birthdays-this-month list, each with a one-tap "contact" action that records the outreach.

11.18 Events / BEO (/events)

Users: owner.

- **FR-EVT-1** KPI tiles: Live now, Bookings, Pipeline value, Avg guests.
- **FR-EVT-2 New booking** — name, schedule, venue, guests, package, contract value, deposit, and cost lines.
- **FR-EVT-3 Banquet Event Order (BEO) view** — guest/revenue/per-guest stats, cost breakdown, **event P&L** with margin %, deposit vs balance.
- **FR-EVT-4 Record deposit** against an outstanding balance; the first deposit confirms a pending booking.
- **FR-EVT-5** Status flow Deposit pending → Confirmed → Live → Completed; **print BEO**.

11.19 Analytics (/reports)

Users: owner, manager.

- **FR-RPT-1** KPI tiles: Revenue, Gross margin (with COGS), Avg ticket, Items sold.
- **FR-RPT-2 Menu engineering matrix** — classifies items into Star / Puzzle / Plowhorse / Dog by margin vs popularity, with a recommendation per quadrant.
- **FR-RPT-3 Revenue by channel** and **Peak hours** (24-hour order histogram for staffing).
- **FR-RPT-4 Stock leakage** — variance loss + waste cost as a % of revenue.
- **FR-RPT-5 702 Morning Brief** — auto night-audit summary (sales, orders, top item, cash shortage, lateness, open complaints).
- **FR-RPT-6 Branch comparison** (owner only) — today's sales across all branches.
- **FR-RPT-7** Export menu-engineering analysis to CSV.

11.20 Audit Trail (/audit)

Users: owner (group-wide), manager (own branch).

- **FR-ATR-1** KPI tiles: Events logged, Today, Sensitive actions, Staff acting.
- **FR-ATR-2** Filter by category (Sales, Inventory, Transfers, Procurement, Finance, Payroll, HR), free-text search, branch (owner), and a "sensitive only" toggle.
- **FR-ATR-3** Date-grouped timeline; each entry shows action, category, detail, actor, branch, time, amount, severity.
- **FR-ATR-4** Export the filtered log to CSV. Entries are immutable.

11.21 Alerts & Security (/alerts)

Users: owner.

- **FR-ALT-1** KPI tiles: Critical, Warnings, Acknowledged, Active total.
- **FR-ALT-2 Live alert feed** derived from current-branch state: over-pours, low/out stock, expiring/expired batches; alerts can be viewed, acknowledged, and marked all read. Alerts are **grouped by section** — Main Store, Kitchen, Bar, Juice Bar — so each consuming area sees the items relevant to it (low stock by `location` ; over-pours attributed to Bar; expiring batches grouped by the product's line). Sections with no active alerts are hidden.
- **FR-ALT-3** Alert detail explains the anomaly and recommends an action, deep-linking to Inventory.
- **FR-ALT-4 Security posture** checklist (informational): 2FA on admins, RBAC reviewed, audit-log streaming, <24h backup, idle auto-lock.

11.22 HR Dashboard (/hr)

Users: owner, manager. **Purpose:** headcount, scheduling and compliance at a glance.

- **FR-HRDB-1 — KPIs:** Active staff, On shift today, Onboarded (30d), Offboarded (30d); plus Compliance gaps, Certs expiring and Late today panels.
- **FR-HRDB-2 — Today's roster:** active staff scheduled for today, grouped by shift period (Full day / 1st / 2nd / 3rd shift), with on-site indicator and late-mark flag.
- **FR-HRDB-3 — Weekly schedule grid:** rows = active staff, columns = Mon–Sun; each cell shows the assigned shift; today's column highlighted.
- **FR-HRDB-4 — Onboarding / offboarding history:** rolling 30-day lists of new hires and offboarded staff (records retained on file, never deleted).

12. Shared components & utilities

Component / lib	Capability
<code>Modal</code> + <code>ModalButton</code>	Standard dialog: backdrop, Escape-to-close, scroll lock, sizes sm–xl, primary/ghost/danger buttons.
<code>ImportModal</code>	Generic CSV import: template download, file upload or paste, live preview, tolerant parsing, result toast.
<code>ManagerApprovalModal</code>	PIN-gated authorisation with optional reason and <code>ownerOnly</code> restriction.
<code>WasteModal</code>	Log inventory waste with cost calculation and staff/shift attribution.
<code>ShiftBanner</code>	Clock-in (opening float) and clock-out (cash reconciliation + bartender bar count).
<code>lib/csv.ts</code>	RFC-4180-ish CSV parser with quoted-field handling and case-insensitive column lookup.
<code>lib/export.ts</code>	Client-side CSV generation and download.
<code>lib/utils.ts</code>	<code>cn()</code> Tailwind class merge helper.

13. Non-functional requirements

#	Category	Requirement
NFR-1	Performance	First-load JS $\leq \sim 150$ kB per route (current build is within this); interactions feel instant; KPIs derive client-side without blocking.
NFR-2	Responsiveness	Fully usable on tablet and mobile; sidebar collapses to a drawer below 1g. POS and KDS are touch-first.
NFR-3	Availability (production)	Front-of-house functions (POS, KDS) must tolerate intermittent connectivity — an offline/queue mode is required.
NFR-4	Security (production)	Hashed credentials, server-enforced RBAC, encrypted transport, session expiry, idle auto-lock, no secrets in client code.
NFR-5	Auditability	Every money/stock-affecting action is logged immutably with actor, branch, time, and amount.
NFR-6	Data integrity (production)	Stock deduction, shift reconciliation, transfers, and payroll must be transactional and safe under concurrent access.
NFR-7	Accessibility	Keyboard-operable controls, sufficient colour contrast, ARIA labels on icon-only buttons.
NFR-8	Localisation	Currency is Nigerian Naira (₦); tax and statutory rates reflect Nigerian regulation; English UI.
NFR-9	Maintainability	Pages depend only on the <code>useStore()</code> hook surface; the data layer can be swapped without touching pages.
NFR-10	Browser support	Modern evergreen browsers; <code>localStorage</code> required in the prototype.

14. Assumptions & constraints

- **A1** A single restaurant group; one hub + three branches. The design generalises to N branches but multi-tenant (multiple groups) is out of scope.
- **A2** Staff are trusted to the extent their role allows; the audit trail and approvals are the control, not technical lockdown (in the prototype).
- **A3** Nigerian regulatory context: 7.5% VAT, PAYE/Pension/NHF rates as coded. These must be configurable in production.
- **A4** All monetary values are whole Naira; no sub-unit (kobo) handling.
- **A5** The prototype assumes one device per role per branch; concurrent multi-device editing is a production concern.
- **C1** No backend, database, or real auth exists yet — see §15.

- **C2** Hardware (printers, drawers, terminals, scales) is not integrated; "print" uses the browser print dialog.

15. Current state — prototype limitations

The application is **functionally complete as a prototype** but carries deliberate shortcuts that must be resolved before production:

Area	Current state	Risk
Persistence	Browser <code>localStorage</code> only; per-device, clearable, single-user	No shared state; data loss; no real multi-device
Authentication	4-digit PINs hardcoded in <code>STAFF_ROSTER</code> ; session in <code>localStorage</code>	Not secure; PINs visible in client source
RBAC enforcement	Sidebar links filtered only; routes not hard-gated	Direct-URL access bypasses the access matrix
Sentiment analysis	Keyword classifier (<code>sentimentOf</code>)	Crude; misclassifies nuanced feedback
Alerts acknowledgement	Session-local; not persisted	Acknowledgements lost on refresh
Notifications	Derived on render; "read" state session-local	No real push; no history
Reports/figures	Some seeded shifts carry <code>seedSales</code>	Mixed seeded/real data until enough real activity exists
Printing & exports	Browser print + client-side CSV	No formal document templates
Concurrency	None	Last-write-wins; no conflict handling

16. Path to production (recommended roadmap)

Phase 1 — Backend foundation

- Stand up a database and API; port every `useStore()` mutation to a server endpoint, preserving the hook's function surface so pages are untouched.
- Real authentication (hashed credentials, sessions/JWT), server-enforced RBAC per the §8.1 matrix, idle auto-lock.
- Move audit logging server-side and make it genuinely append-only.

Phase 2 — Operational hardening

- Offline/queue mode for POS and KDS so trading survives connectivity loss.
- Real-time sync across devices (websockets) for tickets, tables, transfers, and alerts.

- Transactional stock deduction and shift reconciliation safe under concurrency.
- Hardware integration: receipt/kitchen printers, cash drawer, card terminal, weighing scale.

Phase 3 — Intelligence & integrations

- Replace the keyword sentiment classifier with a proper model.
- Payment-gateway and bank integrations (card, transfer confirmation, payroll disbursement).
- Scheduled/emailed reports (the "702 Morning Brief" already has the UI hook).
- Configurable tax/statutory rates, categories, budgets, and approval tiers (admin settings).

Phase 4 — Scale

- Multi-tenant support (multiple restaurant groups).
- Native mobile apps for managers/owners and for riders.
- Customer-facing ordering and loyalty.

17. Risks & open questions

#	Risk / question	Notes
R1	Routes are not role-gated server-side	Must be closed before any real deployment; currently nav-only.
R2	Inventory <code>reorder</code> is a single par level	Production may need min/max and per-branch pars.
R3	Transfers always originate from the hub	Branch-to-branch transfers are not modelled — confirm if needed.
R4	Payroll rates are hardcoded	Need an admin settings surface; rates change by regulation.
R5	Customer identity is phone-only	Collisions/format variance possible; need normalisation rules.
R6	No refund/partial-refund flow	Voids return stock but there is no post-payment refund path.
Q1	Should managers see other branches read-only?	Today they are fully branch-locked.
Q2	Are branch-to-branch (non-hub) transfers required?	Affects the transfer model.
Q3	What is the source of truth for cash movements between till, wallet, and bank?	Needs a cash-management spec.

18. Success metrics

Metric	Definition	Why it matters
Stock leakage %	$(\text{variance loss} + \text{waste}) \div \text{revenue}$	Core value proposition — should trend down
Shift variance rate	Share of shifts closing non-zero	Measures cash discipline
Over-pour incidents	Negative Bar count variances per period	Bar control
Approval cycle time	Request → decision for transfers/expenses	Manager responsiveness
Transfer dispute rate	$\text{Disputed} \div \text{total transfers}$	Supply-chain integrity
Menu mix health	Share of revenue from Star items	Menu engineering effectiveness
Gross margin %	$(\text{Revenue} - \text{COGS}) \div \text{revenue}$	Profitability
Daily active roles per branch	Roles signing in and transacting	Adoption

19. Appendix

19.1 Development login PINs (prototype only)

Role	Name	PIN	Lands on
owner	Seun O.	0000	/
manager	Tunde A.	1111	/manager-dashboard
cashier	Ada O.	2222	/cashier-home
kitchen	Amara K.	3333	/kitchen-home
bartender	Chukwu B.	4444	/bar-home
storekeeper	Eze M.	5555	/store-home
accountant	Bola F.	6666	/expenses
hr	Funke I.	7777	/hr

19.2 Glossary

Term	Meaning
ROS	Restaurant Operating System
Strong Room	The central warehouse hub; sole origin of stock transfers

Term	Meaning
Hub-and-spoke	Operating model: central hub supplies the branch "spokes"
Waybill	The document/record generated when a transfer is issued
Over-pour	A negative Bar stock variance — drink poured but not sold
Blind receiving	Receiving goods without seeing ordered quantities, to force honest counts
BEO	Banquet Event Order — the operating sheet for an event
Golden record	The single deduplicated customer record, keyed by phone
Imprest / float	A fixed petty-cash level a wallet is topped back up to
COD	Cash on delivery
Plowhorse / Star / Puzzle / Dog	Menu-engineering quadrants (popularity × margin)
Par / reorder level	The stock level at or below which an item counts as Low
KDS	Kitchen Display System

19.3 Module ↔ route ↔ source index

Module	Route	Source file
Login	/login	app/login/page.tsx
Operations Overview	/	app/page.tsx
Manager Dashboard	/manager-dashboard	app/manager-dashboard/page.tsx
Cashier Home	/cashier-home	app/cashier-home/page.tsx
Kitchen Home	/kitchen-home	app/kitchen-home/page.tsx
Bar Home	/bar-home	app/bar-home/page.tsx
Store Home	/store-home	app/store-home/page.tsx
Front of House	/pos	app/pos/page.tsx
Kitchen & Bar Display	/kitchen-bar	app/kitchen-bar/page.tsx
Front of House on Shift	/cashier	app/cashier/page.tsx
Inventory	/inventory	app/inventory/page.tsx
Stock Transfers	/transfers	app/transfers/page.tsx
Procurement	/purchase-orders	app/purchase-orders/page.tsx
Vendors	/vendors	app/vendors/page.tsx
Menu & Recipes	/menu	app/menu/page.tsx

Module	Route	Source file
Expenses & Petty Cash	/expenses	app/expenses/page.tsx
Staff (HR)	/staff	app/staff/page.tsx
HR Dashboard	/hr	app/hr/page.tsx
Payroll	/payroll	app/payroll/page.tsx
Dispatch & Fleet	/dispatch	app/dispatch/page.tsx
Customers (CRM)	/customers	app/customers/page.tsx
Events / BEO	/events	app/events/page.tsx
Analytics	/reports	app/reports/page.tsx
Audit Trail	/audit	app/audit/page.tsx
Alerts & Security	/alerts	app/alerts/page.tsx
Auth / session	—	lib/auth.tsx
Domain store	—	lib/store.tsx
App shell	—	components/AppShell.tsx

20. Stakeholder review — decisions implemented in v1.1

Captured from the Tech4mation product review on **2026-05-22**. Every decision below has been **implemented in the application** and is reflected in the requirements above.

#	Decision	Rationale	Where it landed
D1	Order On hold status — the open-tab model	An order is placed and fires to the kitchen, but the tab stays open so the customer can keep adding to it	Order.status , recordSale(hold) , appendToOrder ; Open tab + amend flow (FR-POS-12/13)
D2	An order is complete only when payment is received	"Sales" must mean money collected; a tab is closed deliberately at the end	closeOrder ; on-hold tabs excluded from revenue (FR-POS-14)
D3	Dashboard: daily sales, total orders, open bar/kitchen tickets, clickable low-stock notification	Confirmed the existing Overview design	Validated — Overview KPIs now count paid orders only
D4	Separate inventory by section — Kitchen, Bar, Juice Bar	Faster, independent audits; sections deplete at different rates	Line type + Inventory filter add "Juice Bar" (FR-INV-10)

#	Decision	Rationale	Where it landed
D5	Menu groups → sub-items ; recent orders visible	Confirmed the existing menu & feed design	Validated — no change needed
D6	Rename " Point of Sale " → " Front of House "	"POS" reads as the card terminal; confused users	Sidebar + page label; route <code>/pos</code> kept
D7	Rename " process order " → " Procurement "	Clearer for staff ordering inventory	Sidebar + page label; route <code>/purchase-orders</code> kept
D8	Rename " Cashier & Shifts " → " Front of House on Shift "	Serves waiters and cashiers alike	Sidebar + page label; route <code>/cashier</code> kept
D9	Shift history with dates + date drop-down	Reconciliation needs a historical view	FR-SHF-6 — date column + filter on the ledger
D10	Configurable shift periods — single or multiple shifts	Branches operate differently	FR-SHF-7 — period picker on clock-in (Full day / 1st / 2nd / 3rd)

20.1 Note on routes

The renames (D6–D8) change on-screen labels only. URL paths (`/pos` , `/cashier` , `/purchase-orders`) are unchanged, so existing links and bookmarks keep working.

21. Supply-chain & finance review — decisions implemented in v1.3

A further review refined the supply-chain and finance model. All decisions below are **implemented in the application**.

#	Decision	Implementation
D11	The Strong Room sells stock to branches — a receipt is required	Issued transfers produce a printable Strong Room issue receipt (FR-TRF-7)
D12	Kitchen, Bar and Juice Bar each hold their own inventory , fed from the branch Main Store on request	Multi-location inventory + internal stock requests (FR-INV-11/12)
D13	No duplicate goods across the Main Store and the sub-stores	One shared product catalogue; stock rows keyed by branch + location + SKU
D14	Branch purchase orders need management approval	PO Pending Approval → Ordered / Rejected (FR-PO-8); destination is locked to the current branch — branches cannot request products for other branches (FR-PO-9)
D15	Petty cash is approved by an Accountant	New 7th Accountant role with its own login and finance workspace (FR-EXP-4)

21.1 The location model

Each branch has four stock locations — **Main Store, Kitchen, Bar, Juice Bar** (the Strong Room has only a store). Goods flow: **Strong Room** → **branch Main Store** (transfer + receipt) → **Main Store** → **Kitchen / Bar / Juice Bar** (internal stock request) → consumed by Front-of-House sales, which deduct each recipe ingredient from the sub-store matching its line. A product is defined once in the shared catalogue; each location simply holds its own count, so the same goods are never duplicated.

22. HR review — decisions implemented in v1.4

A further review formalised the staff lifecycle and scheduling.

#	Decision	Implementation
D16	Staff can be onboarded and offboarded — never deleted, records retained	<code>Employee.status = "Offboarded"</code> + <code>offboardEmployee / reactivateEmployee</code> mutations; attendance, disciplinary and salary history are kept on file. Offboard captures a reason and date.
D17	A proper way to schedule staffing on shift	<code>Employee.weeklySchedule</code> (Mon–Sun) per staff member, each day assigned a shift period (Off / Full day / 1st / 2nd / 3rd shift); edited from the employee file.
D18	An HR dashboard	New <code>/hr</code> page — KPIs, today's roster grouped by shift, a Mon–Sun schedule grid, and recent onboarding / offboarding (FR-HRDB-1..4).

24. Kitchen prep review — decisions implemented in v1.6

Refines how the Kitchen records goods and waste during prep.

#	Decision	Implementation
D23	When stock arrives, the Kitchen logs it in both units (e.g. 10 pcs and ~30 kg of yam)	<code>InventoryItem.altUnit</code> + <code>altOnHand</code> . Seeded for Yam (kg + pcs), Plantain (pcs + kg), Tilapia (pcs + kg). The Adjust → Receive stock dialog shows a second optional field "Also count in {altUnit}"; the inventory table displays "30 kg ≈ 10 pcs" on these items. Recipe deduction stays on the primary unit; the alt is a reference and is manually re-counted via Adjust.
D24	After prep they scale the waste (e.g. plantain peels from 5 kg of plantain) and can upload a photo	<code>WasteEntry.photoName / photoDataUrl</code> . The Waste modal has a " Take or upload photo " affordance — on mobile it opens the device camera via <code><input type="file" accept="image/*" capture="environment"></code> ; on desktop it falls back to a file picker. A new " Prep trim / peel " reason is the default. Small photos store a data URL so the inventory waste log shows a 48 px thumbnail; larger photos store the filename as a record.
D25	The Kitchen needs its own Record waste action	Kitchen home now has both Request stock and Record waste buttons (matches Bar home), opening the Waste modal with <code>location="kitchen"</code> .

24.1 No-duplicates safeguard — added in v1.7

D13 ("no duplicate goods across the Main Store and the sub-stores") was already enforced by the **data model** — every stock row is keyed by `branch + location + SKU`, sub-stores reuse the catalogue's SKU when stock is requested, and Strong Room transfers never create new product definitions. v1.7 closes the remaining **UI seam**:

- The **New product** button is only rendered on the **Main Store** tab. On Kitchen / Bar / Juice Bar tabs it is replaced by an inline hint: *"Stocked from the Main Store · products are defined there."*
- The **New product** dialog (formerly *New SKU*) does a **case-insensitive name lookup** against `store.inventory` as the user types. If the name already exists anywhere in the catalogue, the dialog:
 1. Shows an amber warning panel naming the existing SKU and its location (e.g. *"'Yam' is already in the catalogue · Tracked as KIT-002 in Kitchen Store · use Adjust → Receive stock to add more, or request it into a sub-store."*).
 2. Disables the **Add product** action.
- The same dialog now offers an **alt unit** dropdown at creation time, so dual-unit goods (yam in kg + pcs, plantain in pcs + kg, etc.) can be set up correctly from the start rather than retrofitted.

Net effect: there is no longer any path for an operator to accidentally create a second "Yam" record by typing it in again, on any branch or sub-store.

23. UX & HR review — decisions implemented in v1.5

A UX-quality pass on the HR experience, applying familiar patterns from established platforms.

#	Decision	Implementation
D19	HR should have their own login	New 8th HR Officer role (Funke I., PIN 7777, roaming) with a dedicated workspace — HR Dashboard, Staff, Payroll, Audit Trail. Adds a teal/indigo identity at the PIN pad.
D20	Upload the documents staff bring (guarantor's form, etc.) — not just tick a box	<code>Employee.compliance</code> is now three <code>ComplianceDoc</code> objects (guarantor / contract / ID card). The employee file shows each as a file row : status icon, filename, size, upload date, and Upload / Replace / Download actions. Accepts PDF / JPG / PNG up to 5 MB; small files store a data URL so they can be downloaded again from the demo.
D21	The schedule editor should be better — inspired by what other platforms do	Redesigned with quick-fill presets (Mon–Fri 1st shift, Mon–Sat Full day, Tue–Sat 2nd shift, etc.) plus per-day chip pickers colour-coded by shift period (Full day = green, 1st = sky, 2nd = amber, 3rd = violet), with a working-days summary — modeled on Deputy / When I Work / 7shifts.
D22	Reduce clutter — follow UX patterns users already know	<code>/staff</code> replaces its dense table with a list-row pattern (avatar + name + role + status chip + key issues + chevron), like contact/people lists in Linear, GitHub and iOS Settings. <code>/hr</code> collapses three separate alert cards into one alerts strip , simplifies the today's-roster cards, and tightens the weekly grid colour scheme.

End of document.